Soapy Referee Technical Reference



Matt Kennedy

Section 1: System Overview

Soapy Referee (SREF) is an evaluation and observability harness for a small LLM agent that answers questions about NBA history by writing SQL against a frozen database. **The harness is the product; the agent is the fixture** — a deliberately simple text-to-SQL agent that exists to be measured.

Purpose: learn eval engineering hands-on by building every part of an eval around one agent: an executable gold set, a result-set comparator, a validated LLM judge, model comparison as experiments, a regression gate, and trace-level observability in three tools. The headline metric is the **silent-failure rate** — answers that are wrong, raise no error, and look plausible.

What it found is in the eval report (sref_eval_report.pdf); this document is the system reference.

Design Principles

- **The data is never cleaned.** The agent queries the raw tables. Duplicated games, mixed column types, era-dependent NULLs and the wide home/away layout are the measuring instrument.
- **Nothing from the eval leaks into the agent.** Gold SQL, the trap catalog and query hints never reach its prompt. The agent's schema docs state facts — grain, encodings, observed values, coverage, join keys — never how to query.
- **Gold is executed, not stored.** Gold SQL runs at eval time. Result sets are compared, never SQL text: any query that returns the right rows is right.
- **One path to the data.** Gold and agent SQL run through the same read-only DuckDB layer against the same checksummed snapshot.
- **The run record is the source of truth.** Every run writes its settings and one row per case attempt to `runs/`. Traces, grades, experiments and the report are views over it.
- **Every variable is config.** Model, provider, temperature, prompt version, schema mode, attempts and retries are run settings, recorded with every run.
- **The judge grades faithfulness only,** never correctness, and is itself validated against human labels.
- **Pure Python, no framework.** Each step is a thin named function, so tracing decorates the code rather than restructuring it.

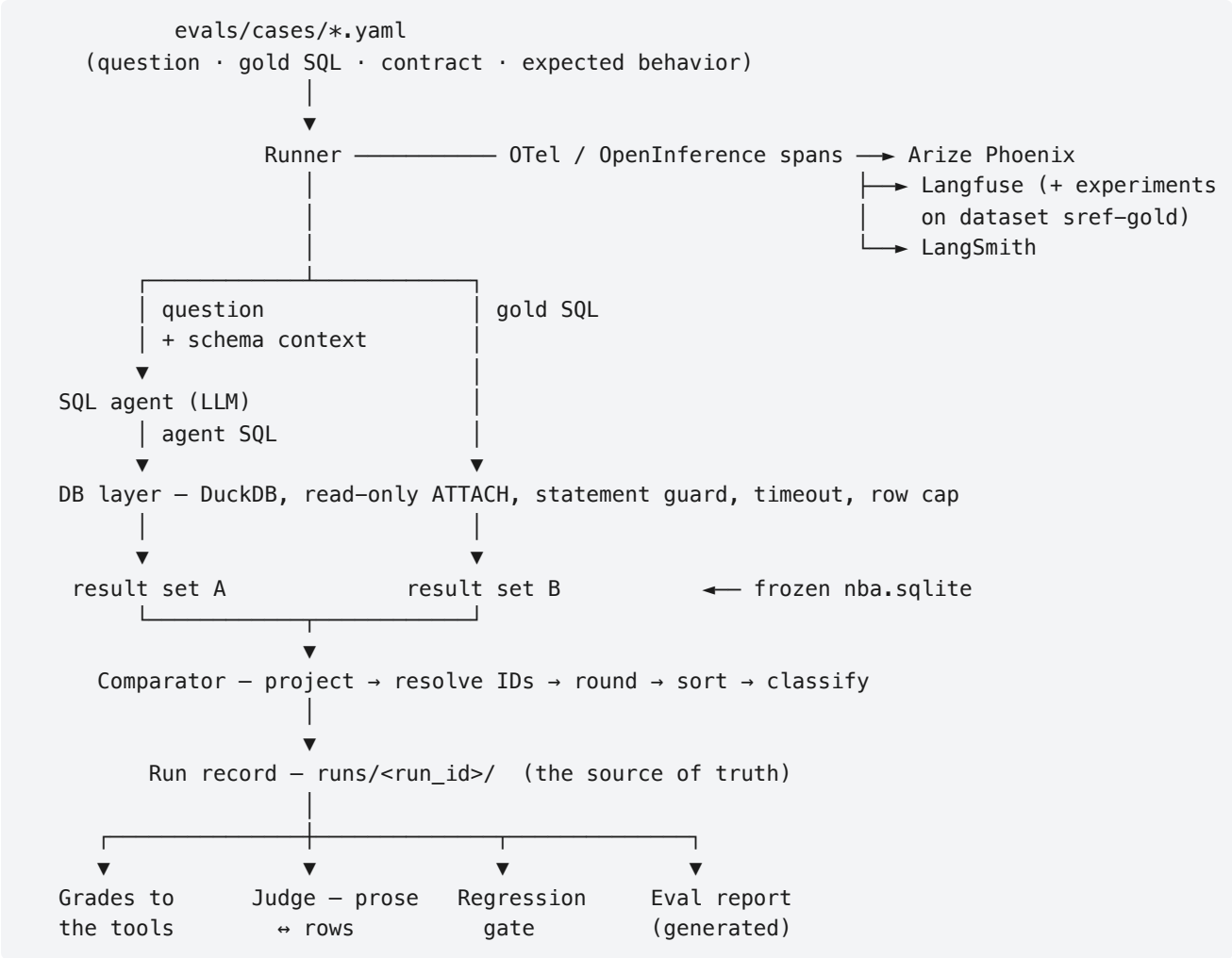
Tool Stack

Layer	Tool	Purpose
Data snapshot	SQLite (`nba.sqlite`)	Frozen NBA database, SHA-256 pinned — ground truth for every case
Query engine	DuckDB + `sqlite` extension	Read-only `ATTACH` ; one execution path for gold and agent SQL
Environment	Python 3.12, uv	Dependencies and reproducible runs
Agent model	`qwen2.5-coder:14b` via Ollama	Local development model, temperature 0
Comparison model	`gpt-6.1-sol` via OpenAI	Closed model for the Phase 7 experiment
Judge model	`gemma4:12b` via Ollama	Faithfulness of the prose answer to its rows
Instrumentation	OpenTelemetry + OpenInference	One span layer, one exporter per backend
Observability	Arize Phoenix (local Docker)	Traces; grades as annotations
Observability	Langfuse v4 (self-hosted Docker)	Traces, scores, human labelling, dataset and experiments
Observability	LangSmith (cloud, free tier)	Traces; grades as feedback
Reports	pandoc + Chrome headless	Markdown → PDF for the eval report and this reference
Tests	pytest, ruff	~400 tests, including every gold case against the snapshot
Database client	DBeaver	Hands-on data discovery against the snapshot (read-only)

Section 2: Architecture & Data Flow

SREF runs in two parts: a **run** (the runner executes every case attempt and writes the run record, sending traces as it goes) and **post-run steps** that read the run record (grades to the tools, the judge, the regression gate, the report).

Eval Loop



One Run, Step by Step

1. **Load the config** (`evals/configs/<name>.yaml`) and, with `--experiment`, check that every case has an item in the Langfuse dataset.
2. **Build the schema block** for each attempt: bare columns or documented schema, with the table order shuffled on attempts 2 and later.

3. **Preflight:** send the schema block once, so a model that is missing, a key that is wrong, or a prompt that does not fit stops the run before it starts.
4. **For each attempt, for each case** — one trace per case attempt:
    - `build_prompt` → `generate_sql` (one model call) → parse the reply as SQL or a refusal
    - `execute_sql.gold` and `execute_sql.agent` through the same DB layer; with `sql_retries`, a failed agent query goes back to the model with the database's error
    - `compare_rows` → a verdict; with `write_answer`, `generate_answer` writes a prose answer from the agent's rows
    - one row appended to the run record
5. **Finish:** the header gains verdict counts and a per-case stability summary.

Post-Run Steps

Step	Reads	Writes
`evals.scores`	Run record	`verdict` and `pass` grades on each trace, in all three tools
`evals.judge`	Rows with a prose answer, plus a labelled set	`judgements.jsonl`; agreement with the labels; optionally grades in Langfuse
`evals.gate`	The run and its pinned baseline	`gate.json`; exit code 0 / 1 / 2
`evals.report`	The report's runs	`docs/sref_eval_report.md`, then a PDF

Build Phases

Phase	What it added
0 — Foundation	Repo, snapshot, data dictionary, hands-on discovery
1 — DB layer + comparator	The measuring instrument, before the thing it measures
2 — Gold set	10 cases with gold SQL, check queries and contracts
3 — Agent + runner	The fixture, the loop, and the run record
4 — Observability	Tracing to three tools; a field-by-field walkthrough
5 — Error analysis	Labels, a failure taxonomy, one measured fix
6 — Judge	Prose answers, a faithfulness judge, agreement with human labels
7 — Experiment + gate	qwen vs `gpt-6.1-sol` as Langfuse experiments; the regression gate
8 — Eval report	Generated from the run record

Section 3: Component Deep Dives

3.1 DB Layer — `evals/db.py`, `evals/snapshot.py`

Purpose: the only path to the data. Gold SQL and agent SQL both run here, so a difference in verdict is a difference in answer and nothing else.

Inputs: `data/nba.sqlite` (path from `NBA_SQLITE_PATH`, default `data/`); `evals/snapshot.yaml`, the snapshot's fingerprint.

Outputs: `QueryResult` — columns, rows, whether the row cap was hit, elapsed time.

Key behavior:

- DuckDB `ATTACH` of the SQLite file, `READ_ONLY`, as schema `nba`
- The file's SHA-256 is checked on connect; a changed snapshot fails loudly, because every gold answer would be invalid
- One statement per call, read-only statements only; a 30-second timeout (DuckDB has none, so a timer interrupts the connection) and a 10,000-row cap
- Read-only introspection (`PRAGMA`, `duckdb_*` functions) stays available to the agent

---

3.2 Gold Cases — `evals/cases/*.yaml`, `evals/cases.py`, `evals/resolve.py`

Purpose: the test set. One YAML file per case; the loader validates every field.

Inputs: case files. Fields: `id`, `question`, `tier` (`baseline` · `trap` · `hard` · `refusal`), `traps`, `expects` (`rows` or `refusal`), `cardinality`, `gold_sql`, `check_sql`, `contract`, `provenance`, `notes`.

Outputs: `Case` objects for the runner, the dataset sync and the report.

Key behavior:

- Tests run every case's gold SQL and check query against the snapshot: wrong row counts, failing gold, or disagreement between the two queries fail the suite
- `resolve.py` turns team names, abbreviations, player names and season labels into ids before comparing, so a team name and a team id can match. Which columns hold teams, players and seasons is part of each case's contract
- Writing rules and coverage: `docs/gold_cases.md`

3.3 Comparator — `evals/comparator.py`

Purpose: turn two result sets into a verdict. Correctness is a result-set question, never a SQL-text question.

Inputs: gold rows, agent rows, the case's `Contract` (ordered, decimals, numeric-string coercion, extra columns allowed, entity columns).

Outputs: `Comparison` — verdict, detail, and the column alignment used.

Key behavior:

- Column names are presentation and are never compared; agent columns are aligned to gold columns by value (trying permutations up to seven columns). Extra agent columns are allowed; every gold column must be present
- Verdicts, strictest first: `exact`, `tolerance` (equal once rounded to the contract's decimals), `unordered` (only when the case wants an order), `superset`, `subset`, `wrong`, `error`
- `superset` and `subset` are reported, never counted as correct
- Known limit: a right answer in another unit (percentage points against a fraction) finds no matching column and scores `wrong`
- Known limit: an empty agent result scores `subset` and counts as a silent failure, though it is easy to spot

---

3.4 Agent — `agent/schema_context.py`, `agent/schema_docs.yaml`, `agent/prompt.py`, `agent/prompts/`

Purpose: the system under test: question and schema in, SQL or a refusal out.

Inputs: the question; the schema block for the run's `schema_mode`; the prompt version.

Outputs: a `Prompt` (system and user text); after the call, a parsed `Reply` — SQL, or a refusal.

Key behavior:

- `schema_mode: columns` renders the bare column list; `docs` adds the hand-written schema docs: table purpose and grain, column meanings, value encodings, observed values, coverage, join keys. Facts only — subject to the no-leak rule
- `agent/build_schema_docs.py` generated the column catalog from the snapshot; descriptions were written by hand
- Prompts are versioned files: `prompts/v1.md` (SQL), `prompts/retry/v1.md` (a failed query and its error), `prompts/answer/v1.md` (a prose answer from the rows, nothing else)
- The reply contract: SQL, or a first line starting `CANNOT ANSWER` to decline. Only the first line is read for the refusal marker

---

3.5 Model Providers — `agent/llm.py`, `agent/ollama.py`, `agent/openai_api.py`, `agent/completion.py`

Purpose: one door to a model, routed by the config's `provider`, so swapping models is a config change.

Inputs: the run config; system and user text.

Outputs: `Completion` — text, prompt and output tokens, reasoning and cached tokens, elapsed time, stop reason.

Key behavior:

- **Ollama:** every call sends `truncate: false`, so an oversized prompt is an error rather than silently losing the schema; preflight checks `num_ctx` against the model's real context length
- **OpenAI:** Chat Completions over the standard library, key from `OPENAI_API_KEY`. Reasoning models accept only temperature 1 and count hidden reasoning against `max_completion_tokens`; rate limits and server errors are retried with backoff, running out of credit is not
- A stop reason of `length` marks a cut-off reply; `reasoning_tokens` explains an empty one

---

3.6 Runner and Run Config — `evals/runner.py`, `evals/config.py`, `evals/configs/`

Purpose: run the gold set against one config and write down what happened.

Inputs: a run config — `name`, `provider`, `model`, `prompt_version`, `schema_mode`, `temperature`, `seed`, `repeats`, `perturbation`, `max_output_tokens`, `num_ctx`, `sql_retries`, `write_answer`, `notes`. Unknown fields are rejected.

Outputs: a run folder in `runs/`; traces; with `--experiment`, one Langfuse experiment per attempt.

Key behavior:

- Cases run attempt by attempt, so each shuffled schema block pays one cold model call, not one per case
- Repeats at temperature 0 without a perturbation are rejected: every attempt would replay the first
- One failing case never ends the run; it is recorded as a runner error
- Shipped arms: `local-columns`, `local-docs` and their `-r3` versions; `local-docs-r3-retry`, `local-docs-r3-answer`; `openai-docs-r3`

3.7 Run Record — `evals/record.py`

Purpose: the durable artifact: committable, diffable, queryable across runs. The tracing tools are views over it.

Outputs: `runs/<run_id>/header.json` and `cases.jsonl` (see Section 4).

Key behavior:

- Written as the run goes: a crash leaves every finished row on disk
- The header carries the full config, snapshot SHA, git commit (`-dirty` when uncommitted), environment (`SREF_ENV`), table order per attempt, verdict counts and stability
- Each row carries the first five rows of both answers, so a person can read "NULL against 117.2" without re-running anything

---

3.8 Tracing — `evals/tracing.py`

Purpose: see every run in Phoenix, Langfuse and LangSmith from one instrumentation layer.

Inputs: backend settings in `.env`; a backend is on when its settings are present.

Outputs: one trace per case attempt, named `<case> #<attempt>`, with spans `build_prompt`, `generate_sql`, `execute_sql.gold`, `execute_sql.agent`, `compare_rows` and, when enabled, `generate_answer`.

Key behavior:

- OpenInference attributes, plus each tool's own keys where it ignores the standard ones (LangSmith tags, Langfuse metadata, release and version)
- The join key — `run_id`, `case_id`, `attempt`, `config` — and `session.id` on every span
- The trace carries the config, model and provider as tags; LLM spans carry token counts including cached and reasoning tokens. Tools price calls themselves; SREF never sends cost
- With `--experiment`, every span carries the Langfuse experiment and item ids, and the case's root carries the gold rows from that run as the expected output
- The full field-by-field wiring, and the lessons behind it: `docs/observability_tools.md` §6

3.9 Grades in the Tools — `evals/scores.py`, `evals/api.py`

Purpose: trace status says whether code threw, not whether the answer was right. This sends each verdict to each tool as its native grade.

Inputs: a run's rows with trace and span ids.

Outputs: two grades per attempt — `verdict` (categorical) and `pass` (boolean) — as LangSmith feedback, Langfuse scores and Phoenix annotations, on the case's root span.

Key behavior:

- Sent after the run, from the run record; re-sending is safe, since each grade's id is built from its span and name
- `api.py` holds the HTTP helper and Langfuse credentials for modules that call a tool's REST API

3.10 Judge — `evals/judge.py`, `evals/judge_prompts/`, `evals/configs/judge/`

Purpose: grade the agent's prose answer for faithfulness to its rows — what the comparator cannot.

Inputs: the question, the rows the answer was written from, the answer. Never the gold answer. A labelled set (`judge_set.jsonl`) in the run folder, including planted unfaithful answers.

Outputs: `judgements.jsonl`; agreement with the human labels, split into lenient and strict misses; with `--send`, `faithful_judge` and `faithful_human` scores in Langfuse.

Key behavior:

- `gemma4:12b` at temperature 0, with a 4,096-token cap because its hidden reasoning counts against it
- Unreadable verdicts are counted separately, never as passes

---

3.11 Dataset and Experiments — `evals/dataset.py`, `runner --experiment`

Purpose: compare two setups the way teams do: the same items, each setup as an experiment, side by side.

Inputs: the gold cases.

Outputs: Langfuse dataset `sref-gold`, one item per case keyed by case id; experiments named `<run_id> #<attempt>`.

Key behavior:

- Items hold the question only; the expected answer travels with each experiment trace, from the gold SQL just run

- Langfuse v4 builds experiments from attributes on the traces, not from a link made afterwards; one experiment per attempt, because Langfuse expects one trace per item
- Items with no matching case are reported, never deleted

3.12 Regression Gate — `evals/gate.py`, `evals/configs/gate/`

Purpose: fail a run that has slipped below its baseline.

Inputs: a run, and the baseline pinned for its config in `configs/gate/default.yaml`.

Outputs: `gate.json` in the run folder; exit code 0 (passed), 1 (failed), 2 (cannot compare).

Key behavior:

- **Score line:** the pass rate must not fall more than `margin_points` (10) below the baseline's
- **No lost stable case:** a case that passed every baseline attempt must not fail every attempt
- Refuses to compare across snapshots, case sets or attempt counts; reports which settings changed

---

3.13 Metrics and Report — `evals/metrics.py`, `evals/report.py`, `tools/build_pdf.sh`

Purpose: the eval's metrics defined once, and the eval report built from them.

Inputs: the runs named in `evals/configs/report/default.yaml`, with model prices; the narrative template `docs/report/sref_eval_template.md`.

Outputs: `docs/sref_eval_report.md`, then `docs/sref_eval_report.pdf`.

Key behavior:

- `metrics.py` is the code behind `docs/eval_plan.md` §7: pass rate, silent and loud failures, pass^k and pass@k, stability, latency, cost
- Every table of numbers in the report is generated; the prose is hand-written in the template. An unknown placeholder stops the build
- `build_pdf.sh`: pandoc (Markdown → self-contained HTML) → Chrome headless → PDF, styled by `docs/report/report.css`

Section 4: Data Reference

The Snapshot

`nba.sqlite` from the Kaggle dataset `wyattowalsh/basketball` (stats.nba.com, CC BY-SA 4.0): 2.2 GB, 16 tables, no indexes, no declared keys. Games from 1946-11-01 to 2023-06-12; play-by-play from 1996-97. SHA-256 `65b6412234d2d8817678d04f7cd2548aec82fec02b6c5b86679cade7572c21c7`.

Table	Rows	Grain
`game`	65,698	game, home and away side by side
`play_by_play`	13,592,899	event within a game
`line_score`	58,053	game, per-quarter scoring, home and away
`game_info` · `game_summary`	58,053 · 58,110	game (attendance, broadcaster)
`other_stats`	28,271	game, 1996 onward
`player` · `common_player_info`	4,815 · 3,632	player
`team` · `team_details` · `team_history`	30 · 27 · 50	franchise
`draft_history` · `draft_combine_stats`	8,257 · 1,633	player × draft
`inactive_players` · `officials`	110,191 · 70,971	player or official × game
`team_info_common`	0	empty

Table profiles, key checks and the trap catalog: `docs/data_dictionary.md` (eval-side only; never agent context).

Run Folder

File	Written by	Holds
`header.json`	runner	Config, snapshot SHA, git commit, environment, table orders, verdict counts, stability, experiment names
`cases.jsonl`	runner	One row per case attempt (below)
`taxonomy.jsonl` · `labels_draft.jsonl`	error analysis	Failure labels and types (Phase 5 run)
`judge_set.jsonl` · `judgements.jsonl`	judge	Human labels and judge verdicts (Phase 6 run)
`gate.json`	gate	The gate's checks against the baseline

Case Row — `cases.jsonl`

Field	Meaning
`case_id` · `tier` · `expects` · `attempt` · `question`	Which case attempt
`verdict` · `detail`	The comparator's verdict and why
`sql` · `refusal` · `raw_response`	What the model said, raw and parsed
`prompt_tokens` · `output_tokens` · `reasoning_tokens` · `cached_tokens`	Token counts; reasoning is part of output, cached part of prompt
`done_reason` · `model_elapsed_s`	Stop reason (`length` = cut off) and model time
`gold_rows` · `agent_rows` · `gold_preview` · `agent_preview` · `query_elapsed_s`	Both answers: row counts, first five rows, query time
`error`	Error text for `error` verdicts
`retries` · `first_sql` · `first_error`	With `sql_retries` : the first failed query and how many retries ran
`answer` · `answer_elapsed_s`	With `write_answer` : the prose answer
`trace_id` · `span_id`	The trace, as OTel hex ids — the join key to every tool

Section 5: Operations

5.1 Standard Startup

Docker Desktop must be running for Phoenix and Langfuse; Ollama runs on the host.

```
cd sref                                    # the repo root
uv sync                                    # environment
cd infra && docker compose up -d && cd ..  # Phoenix + Langfuse
ollama serve                               # or the menu-bar app
```

Health checks:

```
curl -s -o /dev/null -w "%{http_code}\n" http://localhost:6006/   # Phoenix -> 200
curl -s http://localhost:3100/api/public/health                   # Langfuse ->
        {"status":"OK",...}
ollama list                                                       # qwen2.5-coder:14b,
        gemma4:12b
```

Service	URL
Phoenix	`http://localhost:6006`
Langfuse	`http://localhost:3100` (login in `infra/.env`)
LangSmith	smith.langchain.com, project `soapy-referee`
Ollama	`http://localhost:11434`

Keys and which `.env` holds each: `docs/infra.md`. The root `.env` holds the tool keys and `OPENAI_API_KEY`; `infra/.env` holds Langfuse's own secrets.

5.2 Commands

Command	What it does
`uv run pytest`	All tests, including every gold case against the snapshot
`uv run python -m evals.runner --config local-docs`	One pass of the gold set (dev)
`SREF_ENV=eval uv run python -m evals.runner --config local-docs-r3`	A run that counts: three attempts, tagged `eval`
`… --config openai-docs-r3 --experiment`	The comparison arm, recorded as Langfuse experiments
`… --case <case_id>` · `--no-trace` · `--quiet`	One case; no traces; less output
`uv run python -m evals.scores --run <run_id>`	Send the run's grades to the tools
`uv run python -m evals.judge --run <run_id> [--send]`	Judge the prose answers of a `write_answer` run
`uv run python -m evals.dataset`	Sync the gold cases to Langfuse dataset `sref-gold`
`uv run python -m evals.gate --run <run_id>`	Regression gate against the pinned baseline
`uv run python -m evals.report`	Regenerate the eval report's Markdown
`tools/build_pdf.sh <in.md> <out.pdf>`	Markdown → PDF

5.3 Typical Workflows

Try a change. Make it a config setting, run the dev config on the cases it targets (`--case`), then run the `-r3` config with `SREF_ENV=eval` and compare against the noise.

Compare two setups. `evals.dataset` once; both arms with `--experiment` and `SREF_ENV=eval`; `evals.scores` for each run; compare in Langfuse → Experiments, one attempt against the other.

Gate a change. `evals.gate --run <run_id>`. To accept a new normal, pin the run as the config's baseline in `evals/configs/gate/default.yaml`.

Rebuild the report. Edit `docs/report/sref_eval_template.md` (never the generated Markdown), then `evals.report` and `build_pdf.sh`.

5.4 Containers and Data

Container	Purpose
`sref_phoenix`	Phoenix UI and OTLP collector (6006, 4317)
`sref_langfuse_web` · `sref_langfuse_worker`	Langfuse UI, API and ingestion (3100)
`sref_langfuse_postgres` · `sref_clickhouse` · `sref_redis` · `sref_minio`	Langfuse storage; no host ports

`docker compose stop` keeps trace data; `docker compose down -v` deletes it. The Langfuse stack uses about 4 GB of memory at idle, and ClickHouse keeps using CPU while its own system logs grow; stop it when not in use. LangSmith's free tier deletes traces after 14 days — export them first if they matter.

5.5 Common Issues

`cannot reach http://localhost:11434 … Is ollama serve running?`

- Ollama stopped (a closed terminal, or the menu-bar app quit). Start it again; `ollama list` should show the models

`num_ctx … exceeds what … supports` or a context overflow at preflight

- The schema block doesn't fit the configured window. Raise `num_ctx` within the model's limit; never let Ollama truncate

OpenAI: `'temperature' does not support 0 with this model`

- Reasoning models accept only temperature 1. Set `temperature: 1.0` in the config so the run record says what actually ran

OpenAI reply is empty with `done_reason: length`

- Hidden reasoning used the whole `max_output_tokens`. Raise the cap (`openai-docs-r3` uses 4,096); `reasoning_tokens` shows how much went to reasoning

`experiment: … has no item for …`

- A case is missing from the Langfuse dataset. Run `uv run python -m evals.dataset`, then the run again

Langfuse API returns 404 "not available … in Langfuse v4 events_only mode"

- The v1 read endpoints (`/traces`, `/observations`, `/sessions`, dataset runs) are retired. Read through `/api/public/v2/observations`, `/experiments` and `/experiment-items` (the last two need `fromStartTime`)

Langfuse session shows $0.00 cost

- Traces from before `session.id` was set on every span; the session total only sums spans that carry it

Snapshot checksum mismatch on connect

- `data/nba.sqlite` is not the frozen file. Every gold answer depends on it — restore the original; never modify, move or convert it

`gate: cannot compare`

- The run and baseline differ in snapshot, cases or attempts, or the run is its own baseline. Exit code 2 is not a failure of the change, but of the comparison